

CO2 AT1

Student Name	:	Shaik Ahamed Ali
Registration Number	:	192425106
Course Code & Name	:	MLA0302 - Reinforcement Learning
Date of Submission	:	25-07-2026

EXPERIMENT - 1

Delivery Route Optimization

Aim :

To implement a simple Reinforcement Learning (RL) model that helps a delivery vehicle learn the shortest and most rewarding route to deliver packages efficiently.

What is the Problem?

A logistics company (such as Amazon, Flipkart, or DHL) delivers many packages every day. The driver must choose the best route to reach the destination.

However, there are problems such as:

- Heavy traffic
- Road construction
- Longer routes
- Fuel consumption
- Delivery delays

If the driver always follows a fixed route, it may not always be the best option.

Solution

Reinforcement Learning allows the delivery vehicle (agent) to **learn from experience**. After making many trips, it automatically discovers the shortest and fastest delivery route.

What is Reinforcement Learning?

Reinforcement Learning (RL) is a type of Machine Learning where an **agent learns by interacting with an environment**.

The agent:

1. Takes an action.
2. Receives a reward or penalty.
3. Learns which action is best.
4. Repeats until it finds the optimal solution.

Example:

- Correct path → Reward
- Wrong path → Penalty

Over time, the agent learns the best route.

RL Components :

Component	Description
Agent	Delivery Vehicle
Environment	Road Network
State	Current delivery location
Action	Move Left, Right, Up, Down
Reward	+100 for reaching destination, -1 for every move
Policy	Best route chosen by the agent

CODE :

```
#1
import random

route1 = input("Enter the 1st route name: ")
route2 = input("Enter the 2nd route name: ")
route3 = input("Enter the 3rd route name: ")

routes = [route1, route2, route3]
reward = [0, 0, 0]

for i in range(10):
    choice = random.randint(0, 2)

    if choice == 0:
        reward[choice] += 10
    elif choice == 1:
        reward[choice] += 6
    else:
        reward[choice] += 8

print("Rewards:", reward)

best = reward.index(max(reward))
print("Best Route:", routes[best])
```

OUTPUT :

```
Enter the 1st route name: ROUTE 1
Enter the 2nd route name: ROUTE 2
Enter the 3rd route name: ROUTE 3
Rewards: [30, 30, 16]
Best Route: ROUTE 1
```

EXPERIMENT - 2

Fraud Detection System

Aim :

To implement a simple Reinforcement Learning (RL) model that helps a fintech system detect fraudulent transactions by learning from previous transaction patterns.

What is the Problem?

Banks and online payment applications process thousands of transactions every day. Some transactions are genuine, while others may be fraudulent.

Common problems include:

- Unauthorized transactions
- Credit card fraud
- Identity theft
- Financial loss
- False fraud alerts

If the system uses only fixed rules, it may fail to detect new fraud patterns.

Solution :

Reinforcement Learning allows the fraud detection system (agent) to learn from previous transactions. It receives rewards for correctly detecting fraud and penalties for incorrect decisions. Over time, it improves its fraud detection accuracy.

Over time, the system becomes more accurate.

RL Components :

Component	Description
Agent	Fraud Detection System
Environment	Banking Transactions
State	Current Transaction Details
Action	Approve, Reject, Verify Transaction
Reward	+100 for correct fraud detection, -50 for wrong decision
Policy	Select the action with the highest reward

CODE :

```
#2
transaction1 = int(input("Enter the 1st transaction amount: "))
transaction2 = int(input("Enter the 2nd transaction amount: "))
transaction3 = int(input("Enter the 3rd transaction amount: "))

transactions = [transaction1, transaction2, transaction3]

reward = 0

for t in transactions:
    if t > 10000:
        print(t, "- Fraud Detected")
        reward += 10
    else:
        print(t, "- Transaction Approved")
        reward += 2

print("Total Reward:", reward)
```

OUTPUT :

```
Enter the 1st transaction amount: 20000
Enter the 2nd transaction amount: 100000
Enter the 3rd transaction amount: 289777
20000 - Fraud Detected
100000 - Fraud Detected
289777 - Fraud Detected
Total Reward: 30
```

EXPERIMENT - 3

Streaming Platform Recommendation

Aim :

To implement a simple Reinforcement Learning (RL) model that recommends the best content to users based on their viewing history and feedback.

What is the Problem?

Streaming platforms like Netflix, YouTube, and Spotify have thousands of movies, videos, and songs.

Problems include:

- Too many content choices
- Different user interests
- Changing user preferences
- Low user engagement
- Poor recommendations

A fixed recommendation system may not satisfy every user.

Solution :

Reinforcement Learning helps the recommendation system learn from user feedback. It recommends content, observes user reactions, and improves future recommendations.

RL Components :

Component	Description
Agent	Recommendation System
Environment	Streaming Platform
State	User Profile and Watch History
Action	Recommend Movie, Song, or Video
Reward	+100 for positive feedback, -30 for skipped content
Policy	Recommend content with the highest reward

CODE :

```
#3
import random
categories = ["Action", "Comedy", "Drama"]
Q = [0, 0, 0]
learning_rate = 0.5
epsilon = 0.3
user_preference = 0
for episode in range(100):
    if random.random() < epsilon:
        action = random.randint(0, 2)
    else:
        action = Q.index(max(Q))
    if action == user_preference:
        reward = 10
    else:
        reward = -5
    Q[action] += learning_rate * (
        reward - Q[action]
    )
best = Q.index(max(Q))
print("Learned Q-values:", Q)
print("Best Recommendation:", categories[best])
```

OUTPUT :

```
Learned Q-values: [10.0, -4.921875, -4.99755859375]
Best Recommendation: Action
```

EXPERIMENT - 4

Predictive Maintenance

Aim :

To implement a simple Reinforcement Learning (RL) model that predicts equipment maintenance before machine failure.

What is the Problem?

Industries use many machines for manufacturing. Unexpected machine failures can cause production delays and financial losses.

Common problems include:

- Machine breakdown
- High maintenance cost
- Production downtime
- Equipment damage
- Reduced productivity

Traditional maintenance follows fixed schedules, which may not always be efficient.

Solution :

Reinforcement Learning helps the maintenance system learn from machine performance and sensor data. It schedules maintenance only when needed, reducing failures and costs.

RL Components :

Component	Description
Agent	Maintenance System
Environment	Industrial Machines
State	Machine Health and Sensor Data
Action	Perform Maintenance or Continue Operation
Reward	+100 for preventing failure, -50 for machine breakdown
Policy	Choose the maintenance action with the highest reward

CODE :

```
import random
states = ["Healthy", "Warning", "Critical"]
actions = ["Continue", "Inspect", "Maintain"]
Q = [[0, 0, 0],
      [0, 0, 0],
      [0, 0, 0]]
learning_rate = 0.5
epsilon = 0.3
for episode in range(100):
    state = random.randint(0, 2)
    if random.random() < epsilon:
        action = random.randint(0, 2)
    else:
        action = Q[state].index(max(Q[state]))
    if state == 0 and action == 0:
        reward = 10
    elif state == 1 and action == 1:
        reward = 10
    elif state == 2 and action == 2:
        reward = 20
    else:
        reward = -10
    Q[state][action] += learning_rate * (
        reward - Q[state][action]
    )
print("Learned Maintenance Policy:")
for state in range(3):
    best_action = Q[state].index(max(Q[state]))
    print(states[state], "->", actions[best_action])
```

OUTPUT :

```
Learned Maintenance Policy:
Healthy -> Continue
Warning -> Inspect
Critical -> Maintain
```


EXPERIMENT - 5

Transportation System Optimization

Aim :

To implement a simple Reinforcement Learning (RL) model that helps optimize bus scheduling and route allocation by selecting the most efficient transportation route.

What is the Problem?

Public transportation systems operate many buses every day. The transport department must decide the best route and schedule to reduce delays and improve passenger satisfaction.

However, there are problems such as:

- Heavy traffic congestion
- Long waiting times
- Fuel consumption
- Overcrowded buses
- Route delays

If buses always follow fixed schedules and routes, transportation may become inefficient.

Solution :

Reinforcement Learning allows the transportation management system (agent) to learn from previous trips. It selects the best routes and schedules based on traffic conditions and passenger demand.

RL Components :

Component	Description
Agent	Transportation Management System
Environment	Road Network and Traffic
State	Current traffic and passenger demand
Action	Select bus route or schedule
Reward	+100 for on-time arrival, -50 for delay
Policy	Choose the route with the highest reward

CODE :

```
import random

# States (Traffic Conditions)
states = ["Low Traffic", "High Traffic"]

# Actions (Routes)
actions = ["Route A", "Route B"]

# Reward Table
rewards = {
    ("Low Traffic", "Route A"): 10,
    ("Low Traffic", "Route B"): 6,
    ("High Traffic", "Route A"): 4,
    ("High Traffic", "Route B"): 9
}

# Randomly select current traffic condition
state = random.choice(states)

print("Current State:", state)

# Select the best route (Policy)
best_action = max(actions, key=lambda a: rewards[(state, a)])

print("Selected Action:", best_action)
```

OUTPUT :

```
Current State: High Traffic
Selected Action: Route B
```

EXPERIMENT - 6

Dynamic Pricing in E-Commerce

Aim :

To implement a simple Reinforcement Learning (RL) model that adjusts product prices automatically to maximize sales and profit.

What is the Problem?

E-commerce websites such as Amazon and Flipkart sell thousands of products every day.

However, there are problems such as:

- Changing customer demand
- Competitor price changes
- Seasonal offers
- Overstock or low stock
- Profit loss

If prices remain fixed, companies may lose customers or reduce profits.

Solution :

Reinforcement Learning helps the pricing system learn from customer behavior. It increases or decreases product prices based on demand and market conditions.

RL Components :

Component	Description
Agent	Pricing System
Environment	E-Commerce Market
State	Product demand and stock availability
Action	Increase, decrease, or maintain price
Reward	+100 for high profit, -50 for low sales
Policy	Select the price with the highest reward

CODE

:

```
import random

# States (Customer Demand)
states = ["Low Demand", "High Demand"]

# Actions (Pricing)
actions = ["Decrease Price", "Increase Price"]

# Reward Table
rewards = {
    ("Low Demand", "Decrease Price"): 8,
    ("Low Demand", "Increase Price"): 2,
    ("High Demand", "Decrease Price"): 5,
    ("High Demand", "Increase Price"): 10
}

# Randomly choose current demand
state = random.choice(states)

print("Current State:", state)

# Policy: Choose action with maximum reward
best_action = max(actions, key=lambda a: rewards[(state, a)])

print("Selected Action:", best_action)
print("Reward:", rewards[(state, best_action)])
```

OUTPUT :

```
Current State: High Demand
Selected Action: Increase Price
Reward: 10
```

EXPERIMENT - 7

Smart City Waste Management

Aim :

To implement a simple Reinforcement Learning (RL) model that optimizes waste collection routes and schedules for smart cities.

What is the Problem?

Municipal corporations collect waste from many locations every day.

However, there are problems such as:

- Overflowing waste bins
- Unnecessary fuel consumption
- Traffic congestion
- Long collection routes
- High operational costs

If waste collection follows fixed schedules, many bins may remain empty while others overflow.

Solution :

Reinforcement Learning allows the waste management system (agent) to learn from bin fill levels and traffic conditions. It chooses the best collection route and schedule automatically.

RL Components :

Component	Description
Agent	Waste Collection Vehicle
Environment	Smart City Waste Management System
State	Bin fill level and traffic conditions
Action	Select waste collection route
Reward	+100 for efficient collection, -50 for overflow or delay
Policy	Choose the route with the highest reward

CODE :

```
 # Simple RL Waste Management Example

bin_level = 85    # Bin filled (%)

if bin_level >= 80:
    action = "Collect Waste"
    reward = 10
else:
    action = "Skip Bin"
    reward = -2

print("Bin Level:", bin_level)
print("Action:", action)
print("Reward:", reward)
```

OUTPUT :

```
Bin Level: 85
Action: Collect Waste
Reward: 10
```

EXPERIMENT - 8

Network Bandwidth Allocation

Aim :

To implement a simple Reinforcement Learning (RL) model that helps allocate network bandwidth efficiently among users based on network demand.

What is the Problem?

Telecommunication companies provide internet services to millions of users every day. The network must distribute available bandwidth efficiently.

However, there are problems such as:

- Heavy network traffic
- Network congestion
- Slow internet speed
- High latency
- Unequal bandwidth allocation

If bandwidth is allocated using fixed rules, users may experience poor network performance during peak hours.

Solution :

Reinforcement Learning allows the network management system (agent) to learn from network conditions. It dynamically allocates bandwidth based on user demand and network traffic to improve overall performance.

RL Components :

Component	Description
Agent	Network Management System
Environment	Telecommunication Network
State	Current network traffic and bandwidth usage
Action	Allocate or adjust bandwidth
Reward	+100 for efficient allocation, -50 for congestion
Policy	Allocate bandwidth with the highest reward

CODE :

```
 # Simple RL Bandwidth Allocation Example

traffic = 80      # Current network traffic
bandwidth = 100   # Available bandwidth

if traffic < bandwidth:
    action = "Allocate Bandwidth"
    reward = 10
else:
    action = "Reduce Bandwidth"
    reward = -5

print("Traffic:", traffic)
print("Action:", action)
print("Reward:", reward)
```

OUTPUT :

```
Traffic: 80
Action: Allocate Bandwidth
Reward: 10
```


EXPERIMENT - 9

Portfolio Management in Finance

Aim :

To implement a simple Reinforcement Learning (RL) model that helps investors select the best investment by learning from stock market performance.

What is the Problem?

Investors invest in different stocks to earn profits. However, stock prices change every day, making investment decisions difficult.

There are problems such as:

- Market fluctuations
- Investment risk
- Financial losses
- Delayed returns
- Uncertain future prices

Traditional investment models use fixed rules and historical data, which may not always give the best results.

Solution :

Reinforcement Learning allows the investment system (agent) to learn from previous market performance. It selects the best stock based on expected profit and risk, improving investment decisions over time.

RL Components :

Component	Description
Agent	Investor / Investment System
Environment	Stock Market
State	Current stock prices and market conditions
Action	Buy, Sell, or Hold stocks
Reward	+100 for profitable investment, -50 for loss
Policy	Select the investment with the highest reward

CODE :

```
# Portfolio Management using Reward System

# Input expected profit
stock1 = int(input("Enter expected profit of Stock 1 (%): "))
stock2 = int(input("Enter expected profit of Stock 2 (%): "))
stock3 = int(input("Enter expected profit of Stock 3 (%): "))

# Input risk level
risk1 = input("Enter Risk of Stock 1 (Low/Medium/High): ").lower()
risk2 = input("Enter Risk of Stock 2 (Low/Medium/High): ").lower()
risk3 = input("Enter Risk of Stock 3 (Low/Medium/High): ").lower()

# Reward Function
def calculate_reward(profit, risk):
    # Reward based on profit
    if profit >= 20:
        reward = 120
    elif profit >= 15:
        reward = 80
    elif profit >= 10:
        reward = 60
    else:
        reward = 40

    # Reduce reward based on risk
    if risk == "high":
        reward -= 30
    elif risk == "medium":
        reward -= 15
    else:
        reward += 10 # Bonus for low risk

    # Bonus for long-term investment (Delayed Reward)
    reward += 10

    return reward

# Calculate rewards
reward1 = calculate_reward(stock1, risk1)
reward2 = calculate_reward(stock2, risk2)
reward3 = calculate_reward(stock3, risk3)

# Print rewards
print("\nRewards")
print("Stock 1 Reward =", reward1)
print("Stock 2 Reward =", reward2)
print("Stock 3 Reward =", reward3)

# Select Best Stock
if reward1 >= reward2 and reward1 >= reward3:
    print("\nBest Investment: Stock 1")
    print("Expected Profit:", stock1, "%")
    print("Risk:", risk1)
    print("Reward:", reward1)

elif reward2 >= reward1 and reward2 >= reward3:
    print("\nBest Investment: Stock 2")
    print("Expected Profit:", stock2, "%")
    print("Risk:", risk2)
    print("Reward:", reward2)

else:
    print("\nBest Investment: Stock 3")
    print("Expected Profit:", stock3, "%")
    print("Risk:", risk3)
    print("Reward:", reward3)
```

OUTPUT :

```
Enter expected profit of Stock 1 (%): 22
Enter expected profit of Stock 2 (%): 15
Enter expected profit of Stock 3 (%): 35
Enter Risk of Stock 1 (Low/Medium/High): 12
Enter Risk of Stock 2 (Low/Medium/High): 32
Enter Risk of Stock 3 (Low/Medium/High): 15

Rewards
Stock 1 Reward = 120
Stock 2 Reward = 100
Stock 3 Reward = 120

Best Investment: Stock 1
Expected Profit: 22 %
Risk: 12
Reward: 120
```

EXPERIMENT - 10

Renewable Energy Management System

Aim :

To implement a simple Reinforcement Learning (RL) model that helps manage renewable energy sources efficiently by selecting the best energy source based on power generation and environmental conditions.

What is the Problem?

Renewable energy sources such as **Solar, Wind, and Hydro** are used to generate electricity. Their power generation depends on weather conditions.

However, there are problems such as:

- Changing weather conditions
- Uncertain energy generation
- Power fluctuations
- Energy wastage
- Difficulty in balancing energy supply and demand

If a fixed energy source is always selected, power generation may become inefficient.

Solution :

Reinforcement Learning allows the energy management system (agent) to learn from environmental conditions. It selects the best renewable energy source based on weather and power generation, improving energy efficiency.

RL COMPONENTS :

Component	Description
Agent	Energy Management System
Environment	Renewable Energy Sources (Solar, Wind, Hydro)
State	Weather conditions and power generation
Action	Select Solar, Wind, or Hydro energy
Reward	+100 for efficient energy generation, -50 for low power generation
Policy	Choose the energy source with the highest reward

CODE :

```
# Renewable Energy Management using Reward System

# Input power generated
solar = int(input("Enter Solar Power Generated (kwh): "))
wind = int(input("Enter Wind Power Generated (kwh): "))
hydro = int(input("Enter Hydro Power Generated (kwh): "))

# Input weather condition
weather = input("Enter Weather (Sunny/Windy/Rainy): ").lower()

# Reward Function
def calculate_reward(power, source):
    # Reward based on power generated
    if power >= 100:
        reward = 100
    elif power >= 70:
        reward = 80
    elif power >= 40:
        reward = 60
    else:
        reward = 40

    # Environmental uncertainty
    if source == "solar":
        if weather == "sunny":
            reward += 20
        else:
            reward -= 20

    elif source == "wind":
        if weather == "windy":
            reward += 20
        else:
            reward -= 20

    elif source == "hydro":
        if weather == "rainy":
            reward += 20
        else:
            reward -= 20

    return reward

# Calculate rewards
reward1 = calculate_reward(solar, "solar")
reward2 = calculate_reward(wind, "wind")
reward3 = calculate_reward(hydro, "hydro")

# Print rewards
print("\nRewards:")
print("Solar Reward =", reward1)
print("Wind Reward =", reward2)
print("Hydro Reward =", reward3)

# Select best source
if reward1 >= reward2 and reward1 >= reward3:
    print("\nBest Energy Source: Solar")
    print("Power Generated:", solar, "kwh")
    print("Reward:", reward1)

elif reward2 >= reward1 and reward2 >= reward3:
    print("\nBest Energy Source: Wind")
    print("Power Generated:", wind, "kwh")
    print("Reward:", reward2)

else:
    print("\nBest Energy Source: Hydro")
    print("Power Generated:", hydro, "kwh")
    print("Reward:", reward3)
```

OUTPUT :

```
Enter Solar Power Generated (kwh): 100
Enter Wind Power Generated (kwh): 70
Enter Hydro Power Generated (kwh): 80
Enter Weather (Sunny/Windy/Rainy): Sunny

Rewards
Solar Reward = 120
Wind Reward = 60
Hydro Reward = 70

Best Energy Source: Solar
Power Generated: 100 kwh
Reward: 120
```